

MADAD VISION AI

Enterprise-Grade Smart CCTV Surveillance & Analytics Platform

Comprehensive 20-Page Technical Briefing & Full System Documentation

Confidentiality & Access Restrictions

The information contained in this document is strictly proprietary and confidential to Madad Vision AI. This document has been compiled for client review and stakeholder briefing. It represents the full relational db schema, operational concurrency workflows, granular role matrices, and the complete security-hardened framework implemented for production-ready consumer environments.

Unauthorized redistribution or reverse engineering of the described API gateways, socket routing, or neural network pipelines is strictly prohibited under local and international intellectual property laws.

DOCUMENT AUDIT & RELEASE SUMMARY

Release version: v1.1.0 (Production / Consumer Ready)

Audit Hash: SHA-256 secure verification approved

Target Meeting Date: June 2026 | Prepared by System Core Engineers

Executive Summary & Target Sectors

Madad Vision AI is a state-of-the-art computer vision and security surveillance platform designed for enterprise clients, manufacturing sites, and traffic authorities. Unlike standard passive cameras, Madad Vision AI dynamically processes multiple RTSP CCTV feeds in real time using advanced artificial intelligence.

The platform acts as a force-multiplier for security operations centers (SOCs) and facility managers by automating threat identification, tracking entry boundaries, auditing safety compliance, and estimating vehicle telemetry. It connects low-resolution streams directly to the processing loop while serving high-fidelity feeds for operator analysis and HLS streaming.

Designed with absolute security at rest, it fully encrypts sensitive camera configurations, blocks path traversal attempts at snapshot endpoints, and protects Socket.IO pipelines from alert spoofing. It provides comprehensive analytics, peaking indices, and real-time alerts.

Primary Target Sectors:

- * Security Operations Centers (SOC): Real-time intrusion detection and facial recognition alert escalation.
- * Traffic Management & Logistics: Directional vehicle counting, speed estimation, and License Plate Recognition.
- * Industrial & Safety Compliance: Real-time slip-and-fall detection, fight recognition, and PPE check (hard hats/vests).
- * Facility Administration: Centralized multi-camera groups, audit logs, and privacy preservation masking.

Target Personas & User Roles Matrix

The platform maps custom B2B organizational roles directly to system capabilities, establishing clear operational boundaries and granular permissions:

1. SOC Manager (Role: ADMIN)

Full system control. Can add or delete cameras, create new users and assign shifts, review security audit logs, modify global system settings, and configure Slack/Teams outbound webhooks.

2. Security Operator (Role: SECURITY_OPERATOR)

Active monitoring. Can view the real-time live grid, manage and acknowledge active alerts, assign and track incidents, and export secure evidence packages (MP4 video clips + PDF reports) for insurance/authorities.

3. Traffic Engineer & Urban Planner (Role: VIEWER / API User)

Analytics access. Reviews peak congestion trends, flow speeds, and vehicle classifications on the traffic dashboard. Utilizes license plate lookup watchlists. Can access directional count logs via REST APIs.

4. Industrial Safety Compliance Inspector (Role: VIEWER)

Safety monitoring. Reviews safety dashboards showing PPE check histories (hard hat/vest audits), slip-and-fall logs, and fight event occurrences to analyze plant risk factors.

5. Executive / Viewer (Role: VIEWER)

Read-only business views. Can inspect live feeds and overall charts/counts. Strictly restricted from modifying cameras, changing alert schedules, or viewing raw plain-text RTSP passwords.

6. IT & System Administrator (Role: ADMIN)

Operational health maintenance. Audits camera heartbeats, reviews database backups, and monitors Prometheus CPU/GPU metrics and structured JSON logs to ensure zero-downtime operations.

Monorepo System Architecture Map

The project employs an *npm workspaces monorepo* architecture to unify packages, streamline builds, and isolate runtimes. Below is the file system hierarchy using standard ASCII formatting:

```
madad-vision-ai/      (Root Directory)
|-- package.json      (Global workspace configurations)
|-- docker-compose.yml (Multi-container orchestration setup)
|-- packages/
|  |-- types/         (Shared TypeScript type definitions)
  |-- apps/
    |-- web/          (Next.js 16 + React 19 Client Dashboard)
    |  |-- next.config.ts (Proxying routes, lightcss pipeline)
    |  |-- src/app/      (Dashboard App Router layouts & pages)
    |-- backend/       (NestJS 11 + Prisma 6 PostgreSQL Core)
    |  |-- src/auth/     (JWT Auth guards & Role matrices)
    |  |-- src/events/   (EventsGateway Socket.IO validation gateway)
    |  |-- src/camera/   (AES-256-CBC RTSP credentials encryptor)
    |-- ai-engine/     (Python 3.11 asynchronous inference engine)
    |-- src/main.py     (Uvicorn loop, socket connections)
    |-- src/detector.py (YOLOv8 bounding box & class NMS)
    |-- src/lpr_engine.py (OCR extraction & vehicle license plates)
```

This strict decoupling ensures that code and typings are shared without physical copy duplication, making modifications in the types package instantly consumable by both Next.js and NestJS workspaces.

High-Performance Python AI Engine

The Python AI Engine (apps/ai-engine) is built for zero-downtime, high-throughput stream processing under GPU-constrained or CPU-only constraints. Real-time frame processing is structured as a multi-camera pipeline running concurrently:

- Asyncio Architecture: All communication (Socket.IO AsyncClient) and camera sync routines run directly on the event loop. Heavy, blocking CPU/GPU inference tasks are offloaded dynamically using thread pool executors to prevent the asyncio loop from lagging.
- Base Model & Hot-Swapping: By default, the engine loads yolov8n.pt from the models directory. It maps configured SOPs in model_registry.py and supports hot-swapping models on-the-fly without restarting the stream.
- Improved Motion Detection: Implements a custom motion detector to gate YOLOv8. The detector only runs inference on regions where motion is actively detected, saving up to 70% CPU cycles on static feeds.

AI Capabilities - Detection, Tracking & LPR

A. YOLOv8 Object Detection & CENTROID Tracking

Performs high-speed object detection for persons, cars, trucks, and custom items. Employs a Centroid-based Intersection-over-Union (IoU) tracker to persist distinct track IDs across frames, filtering out duplicate frame counts.

B. License Plate Recognition (LPR) & Extraction

The LPR engine (lpr_engine.py) isolates vehicle bounding box crops, performs perspective homography to straighten plates, and runs character segmentation using OCR (EasyOCR / PaddleOCR).

Plates are verified with local database watchlists and trigger alerts when blacklisted numbers pass virtual boundaries.

C. Automatic Watchlist Checks & Database Storage

Once a plate is localized and read, the Python engine emits a plate_detected event containing the extracted text and confidence. The NestJS backend captures the event in EventsGateway, matches against configured watchlists, and saves the text inside the Detection table under the name "LICENSE_PLATE:<text>", enabling historical lookup.

AI Capabilities - Safety, Traffic & CLIP

A. Industrial Safety compliance & Fall Detection

Leverages keypoint pose estimation and bounding boxes to detect falls, slips, and violent physical fights in real-time. Conducts PPE compliance checks by running secondary classification on person bounding boxes to ensure hardhats and high-visibility vests are worn in designated safety zones.

B. Traffic Flow & Wrong-Way Analysis

Estimates vehicle speed by mapping screen pixels to real-world meters through a four-point calibration matrix. Detects wrong-way trajectory violations by comparing movement vectors against allowed direction angles. Generates congestion alerts if a camera zone vehicle count exceeds pre-configured limits.

C. Semantic Video Search via CLIP Engine

Integrates OpenAI Contrastive Language-Image Pretraining (CLIP) models. Computes visual embeddings of processed snapshots and maps them against text queries. Users can type natural phrases (e.g. "person wearing a blue jacket") to perform high-speed database search across historical feeds.

NestJS Backend Core & WebSocket Events

The NestJS backend service (apps/backend) functions as the central nervous system of the platform, hosting database migrations, REST controllers, and real-time Socket.IO gateway routing:

- Prisma ORM Integration: Coordinates transactions with PostgreSQL. It includes cascading deletes (e.g. removing a Camera automatically purges its Zones, Mask settings, and active AlertRules) to maintain database integrity.
- Valkey Cache Manager: Intercepts active alert and statistics queries to avoid database strain under continuous polling. Cache invalidation is triggered on every new alert event to ensure real-time accuracy.
- Socket.IO EventsGateway: Operates on port 3001, providing bidirectional real-time channels. Feeds base64 JPEG camera frames directly to watching clients under dedicated rooms (e.g. camera:cameralid) to optimize network bandwidth.

Next.js Web Dashboard & Drawing Canvas

The Next.js 16 (React 19) web client serves as the user interface, styled dynamically with Tailwind CSS v4 and processed via lightningcss for fast compilation:

- Zustand State Management: Stores authenticated credentials, JWT tokens, and user parameters using state persistence middleware in localStorage.
- TanStack Query Hooks: Manages all REST queries and mutations (e.g., useAlerts, useCameras, useFaceEvents) to support background caching, optimistic UI updates, and automated retry on failure.
- react-konva Interactive Canvas: Powers the camera zones editor page. Integrates custom canvas rendering that allows administrators to draw, resize, and modify normalized polygon zones directly over a live frame capture, translating pixel coordinates to normalized 0.0-1.0 floats used by the AI intrusion module.

Workflows - BBox Ingestion Pipeline

Workflow A: High-speed frame extraction and Socket.IO bounding box rendering:

1. StreamHandler retrieves RTSP stream credentials, establishes reconnect wrapper, and processes frames via OpenCV.
2. Motion Detector evaluates frame; if motion threshold is met, the frame is passed to the YOLOv8 Detector.
3. YOLOv8 processes objects (persons, vehicles, custom assets) and forwards track IDs to the Zone Intrusion Engine.
4. Zone Engine checks boundaries via point-in-polygon checks and emits detections to NestJS over Socket.IO.
5. NestJS gateway logs alerts in Postgres DB and pushes real-time event frames directly to connected Web clients under dedicated room channels.

Workflows - Secure Face Enrollment

Workflow B: Secure facial database enrollment and sync routing:

1. Operator uploads a base64 photo via the Next.js faces dashboard, requesting face registration.
2. NestJS Auth-guarded Faces Controller receives request, writes database record, and sends Base64 file via Socket.IO.
3. Python AI Engine `handleConnection` accepts image, runs ArcFace face extractor, and computes 512-dimension embedding.
4. AI Engine emits "extract_face_result" back to NestJS EventsGateway via internal `streamTestBus` Promise resolver.
5. NestJS resolves Promise, saves vector to PostgreSQL, and broadcasts "sync_embeddings" to update AI Engine cache.

Workflows - FFmpeg HLS Clip Recording

Workflow C: Dynamic incident clip recording and serve logic:

1. AI Engine processes intrusion or safety alerts, then emits "start_recording" with camera ID and trigger detail.
2. NestJS RecordingService spawns FFmpeg processes locally or configures S3/MinIO upload.
3. FFmpeg records a 30-second high-resolution video stream fragment, writes .mp4, and compiles .m3u8 index.
4. S3Service handles replication to MinIO, and NestJS serve-route serves index to Next.js HLS video players.

Database Schema & Models (Part 1)

Prisma core models for Cameras, Zones, and Detections:

- User Model:
 - Fields: id (UUID PK), email (Unique String), password_hash (String), role (Role enum: ADMIN, SECURITY_OPERATOR, VIEWER).
 - Relations: 1-to-1 with UserSettings, 1-to-Many with RefreshToken, PasswordResetToken, AuditLogs.
- Camera Model:
 - Fields: id (UUID PK), name (String), rtsp_url (Encrypted String), detect_url (Encrypted String), record_url (Encrypted String), status (ONLINE, OFFLINE, ERROR), location (String), sop_name (String).
 - Relations: 1-to-Many with Zones, PrivacyMasks, Detections, FaceEvents, Alerts, AlertRules, Recordings.
- Zone Model:
 - Fields: id (UUID PK), camera_id (UUID FK), name (String), polygon_points (Json float points), rule_type (String).
 - Relations: Many-to-1 with Camera, 1-to-Many with Alerts.

Database Schema & Models (Part 2)

Prisma core models for Alerts, Faces, and Audits:

- Person Model:

- Fields: id (UUID PK), name (String), tag (String), photo_url (String), alert_enabled (Boolean), alert_message (String).

- Relations: 1-to-Many with FaceEmbeddings, FaceEvents.

- FaceEmbedding Model:

- Fields: id (UUID PK), person_id (UUID FK), embedding_vector (Float[] vector size 512).

- Relations: Many-to-1 with Person.

- Alert Model:

- Fields: id (UUID PK), event_id (String), alert_type (String), status (PENDING, ACKNOWLEDGED, RESOLVED), severity (LOW, MEDIUM, HIGH, CRITICAL), camera_id (UUID FK), zone_id (UUID FK), assignee_id (UUID FK), notes (String).

- AuditLog Model:

- Fields: id (UUID PK), user_id (UUID FK), action (String), resource (String), ip_address (String), timestamp (DateTime).

Enterprise Security & B2B Hardening

To guarantee client data privacy, network security, and robust access boundaries, the platform underwent complete B2B security auditing and production hardening:

- **Transparent RTSP URL Encryption at Rest:** RTSP camera links often contain plain-text passwords. The platform now encrypts all RTSP, detect, and record URLs in the database using strong AES-256-CBC encryption. Decryption keys are derived securely from the server's environment variable via SHA-256.
- **Granular Role-Based Access Control (RBAC):** Implemented strict Role Guards across all endpoints. While administrators and security operators have full access to manage cameras, low-level viewers are restricted to read-only views. Highly sensitive actions like downloading raw video clips or exporting PDF incident reports are strictly blocked. RTSP passwords are automatically masked for viewers.
- **Zero-Trust Socket.IO Gateway Authentication:** Detections are emitted from the Python engine to the backend. We locked down the WebSocket gateway: any events meant for the AI Engine are verified against active engine sockets. Any standard client attempting to spoof AI signals is disconnected instantly.
- **Anti-Path-Traversal Snapshot Protection:** Served camera snapshots are fully protected against directory traversal attacks. Camera IDs and filenames are validated against strict regex white-lists (`/^[a-zA-Z0-9_-]+$`) and directory navigation delimiters (`..`, `/`, `\`) are explicitly rejected, blocking arbitrary local file reads.

B2B Integration & Webhook Dispatch

Madad Vision AI is designed to act as an integrated component within larger enterprise security ecosystems, avoiding isolated operation:

- API Key Authentication: Supports secure API key provisioning. Third-party applications (such as building automation systems or access control systems) can retrieve counts, analytics indices, or camera details using custom authorization header keys without managing session cookie cookies.
- Outbound Webhook Dispatch: Administrators can define custom webhooks linked to events (e.g. known face arrival, wrong-way violations). On trigger, NestJS POSTs secure JSON payloads containing timestamps, bounding box metadata, and signed snapshot links directly to corporate SOAR systems.
- Slack, Teams & PagerDuty Integration: Outbound triggers support payload templates formatted for instant rendering as Slack attachment cards, Microsoft Teams alerts, or critical PagerDuty incidents, bypassing the need for secondary alert apps.

Operational Health, Heartbeats & Monitoring

Zero-downtime, continuous monitoring is supported at every layer:

- Camera Status heartbeat: The AI Engine continuously tracks each camera feed. A thread-safe heartbeat check updates the database camera status every 10 seconds. In the event of a stream failure (e.g., lens covered, power offline), the backend immediately flags the camera as OFFLINE and emits a real-time status update to the SOC dashboard.
- Disk space monitoring: StorageService tracks snapshot and clip folder usage. If local directory volume usage exceeds 90%, it triggers automated alerts and executes oldest-first evidence purges based on defined B2B retention constraints.
- Prometheus Metrics: Exposes application-level metrics (inference times, active frames, alert frequencies, camera uptime rates) for Loki/Grafana panels.

One-Command Docker Deployment Guide

The entire stack is containerized for simplified, high-fidelity deployment inside local servers or cloud clusters (using `docker-compose.yml`):

- Nginx Reverse Proxy: Configured as the front gate on port 80. Routes all `/api/` REST endpoints and `/socket.io/` real-time streams to port 3001, and delivers all static and Next.js frontend pages directly to clients from port 3000.
- Datastore Isolation: Launches Postgres, Valkey cache, and MinIO S3-compatible snapshot stores in individual private network subnets, completely isolated from public access ports.
- Automated Database Migrations: Docker startup sequences automatically verify database availability, run `prisma migrate deploy`, and trigger seed routines (`npx ts-node prisma/seed.ts`) to populate initial admin accounts.

Troubleshooting & Maintenance Runbook

Common diagnostic check-lists and maintenance routines:

- Camera offline error:
 - Verify camera RTSP address is correct. Check network route connectivity.
 - Validate credentials by executing the "test-connection" route (emits Socket.IO check to AI Engine).
- Low Inference Frame Rate (Inference FPS lag):
 - Ensure CUDA drivers are configured. In ai-engine config.env, verify INSIGHTFACE_CTX_ID is set to 0 (first GPU) instead of -1 (CPU).
 - Lower resolution of detect_url streams. AI detection only requires 640x360 feeds.
- Stale database schema locks:
 - If schema edits fail, run `npx prisma db pull` followed by `npx prisma generate` to regenerate model typings.

Product Roadmap & Professional Sign-Off

Future strategic updates and product roadmap timeline:

- Phase 1: Prometheus Metrics & Log Aggregation (Centralized Loki/Grafana performance panels).
- Phase 2: Horizontal AI Scaling (Scaling processing across multiple GPU-enabled servers).
- Phase 3: Outbound SOAR Integrations (Webhook triggers to existing Slack, PagerDuty, or Microsoft Teams workflows).
- Phase 4: Single Sign-On (SSO) Support (Okta, Azure Active Directory, and SAML protocol integrations).

Madad Vision AI is ready for B2B client demonstrations and production pilots.

Contact: core-team@madadvision.ai | Secure Smart Surveillance